

Documentação

Aplicação Fluxo do Saber: Marketplace Acadêmico Colaborativo

Equipe:

- João Marcos
- Carlos Alberto
- Ycaro Toribio
- Dyogenes Meira
- Pedro Henrique

Objetivo Geral

Criação de uma plataforma digital com foco em mobile-first, estabelecendo um ecossistema de marketplace voltado ao público universitário. O projeto visa simplificar o comércio e a permuta de recursos acadêmicos, integrando os seguintes pilares:

Desenvolvimento Front-end: Arquitetura reativa utilizando React, contemplando componentização modular, gerenciamento de rotas por estado e integração com dados simulados.

Design de Interface (UI/UX): Experiência do usuário otimizada para smartphones através da implementação do framework Tailwind CSS.

Publicação e Nuvem: Disponibilização da aplicação em ambiente de nuvem para acesso funcional via plataforma Vercel/v0.

Regras de Negócio

As regras de negócio representam o funcionamento lógico da aplicação e definem como o sistema deve se comportar:



Produtos e Categorias

- O sistema permite a negociação de materiais educacionais segmentados por categorias:
 - Livros e materiais;
 - eletrônicos;
 - Acessórios;
 - Equipamentos técnicos e Lazer.



Tipos de Transação e Condição

- Os produtos devem ser cadastrados definindo o seu estado (Novo, Seminovo, Usado) e a modalidade de negociação:
 - **Venda:** Transação financeira direta.
 - **Troca:** Permuta de itens entre alunos.
 - **Venda/Troca:** Aceita ambas as modalidades.



Sistema de Propostas e Negociação

- O sistema deve possuir um fluxo de interação entre alunos:
 - O comprador pode enviar uma proposta formal de troca pelo aplicativo.
 - O vendedor pode aceitar ou recusar a proposta.
 - A aplicação conta com um chat interno para negociação e alinhamento de entregas.



Logística e Entrega

- Diferente de e-commerces tradicionais, o Fluxo do Saber não gerencia fretes nem calcula taxas de entrega. A logística de entrega/recebimento ocorre presencialmente nos campi universitários, sendo combinada via chat.



Sistema de avaliação

- O sistema exibe a reputação dos vendedores para garantir segurança à comunidade, variando de 1 a 5 estrelas.
-

Frameworks e Bibliotecas

- **Utilização de React + Next.js:** Para a construção da interface reativa e lógica de navegação baseada em estados
- **Uso de Tailwind CSS:** Para a estilização avançada, garantindo responsividade e aplicação de classes utilitárias no modelo mobile-first.
- **Uso de shadcn/ui:** Para componentes padronizados de interface (como inputs, forms e checkboxes).
- **Uso de Lucide React:** Biblioteca oficial utilizada para a renderização de ícones SVG (coração de favoritos, lupa de busca, sino de notificações).
- **Figma:** Utilizado na fase inicial de ideação para a criação de wireframes e protótipos de alta fidelidade das 18 telas originais.
- **Deploy via v0/Vercel:** Hospedagem em nuvem do código gerado para acesso público do protótipo funcional.

Interface e Responsividade

- **Responsividade mobile-first:** A aplicação foi travada em um *container* de `max-w-[375px]` simulando as dimensões exatas de um celular (estilo iPhone) para garantir a melhor experiência, independente se acessada via desktop ou smartphone.
- **Modos de Visualização:** Implementação estrita de **Light Mode** (Modo Claro) para manter a identidade visual acadêmica e limpa.
- **Paleta de Cores:**
 - Cor Primária: Verde vibrante (`#00C896`) para botões de ação e cabeçalhos.
 - Plano de Fundo Principal: Verde menta/claro (`#CFEFEC`).
 - Textos: Slate escuro (`#0F172A`) para garantir alto contraste e legibilidade.

Componentização

Para garantir legibilidade e um visual moderno, alinhado às diretrizes de UI, a aplicação utiliza as fontes Inter (para o corpo do texto e leitura fluida) e Geist (para títulos e destaques visuais), ambas nativas do ecossistema Tailwind/v0.

Estrutura de Componentes: A interface foi arquitetada de forma modular, dividindo responsabilidades entre componentes reutilizáveis:

Componentes Globais:

- **AppContainer:** O principal e que envolve tudo, restringindo o layout para mobile-first (`max-w-[375px]`).
- **BottomNavigation:** Barra de navegação inferior persistente com o botão flutuante de ação.
- **TopHeader:** Cabeçalho dinâmico que adapta o título e o botão de "Voltar" conforme a tela ativa.

Componentes Gerais (Comprador/Descoberta):

- **AuthFlow:** Gerencia as 4 etapas de autenticação e recuperação de senha.
- **HomeDashboard:** Tela inicial com feed de rolagem horizontal para categorias e recomendações.
- **CheckoutSimulator:** Gerencia o fluxo de resumo, método de pagamento simulado e tela de sucesso.

Componentes do Vendedor:

- **SellerProfile:** Exibe o avatar, reputação e o catálogo de produtos ativos do vendedor.
- **CreateAdForm:** Formulário interativo para publicação de novos itens para venda ou troca.
- **TradeReview:** Interface de comparação lado a lado para o vendedor analisar, aceitar ou recusar propostas de troca.



Persistência de Dados e Operações

A persistência de dados no estágio atual da aplicação foi construída para funcionar localmente, garantindo que as operações ocorram em tempo real e de forma fluida sem a necessidade de um backend externo. Utilizamos as seguintes abordagens no código:

- **Gerenciamento de Estado (React `useState`):** Os dados mutáveis e interações do usuário (como a exclusão de um anúncio, ou a troca de etapas no fluxo de checkout) são retidos ativamente na memória da sessão atual utilizando os estados do React. Quando o usuário executa uma ação, a função atualiza o estado em tempo real, refletindo a mudança na interface instantaneamente.
- **Estruturas de Dados Estáticas (Mock Data):** As informações de catálogos, perfis de usuários e notificações (ex: listas como `INITIAL_ADS`) foram estruturadas em arrays e objetos imutáveis diretamente no código-fonte. Essa abordagem permite simular a persistência de um banco de dados relacional populado, viabilizando o desenvolvimento e o teste de renderização condicional dinâmica nas telas.



Feedback Visual e UX

- **Hover effects:** Efeitos de transição (`transition-colors`) em botões secundários, ícones de notificação e cards de "buscas recentes", que ganham fundos sutis e circulares ao passar o mouse.
- **Feedback visual em botões:** Botões de ação principal (Criar Conta, Entrar, Comprar) mudam para uma tonalidade verde mais escura ao sofrerem interação.
- **Scroll Nativo Mobile:** Ocultação forçada das barras de rolagem nativas

(`scrollbar-hide`) nas listas de produtos e categorias, substituindo pela rolagem de arrasto (drag/swipe) típica de telas touch.

Segurança e Autenticação

- **Jornada de Login:** A aplicação exige autenticação, possuindo fluxos estruturados de *Login*, *Cadastro de Usuário* e *Recuperação de Senha*.
- Os formulários possuem validação visual implícita, incluindo alternância de visibilidade da senha (ícone de olho) e aceitação obrigatória de Termos de Uso nativa.

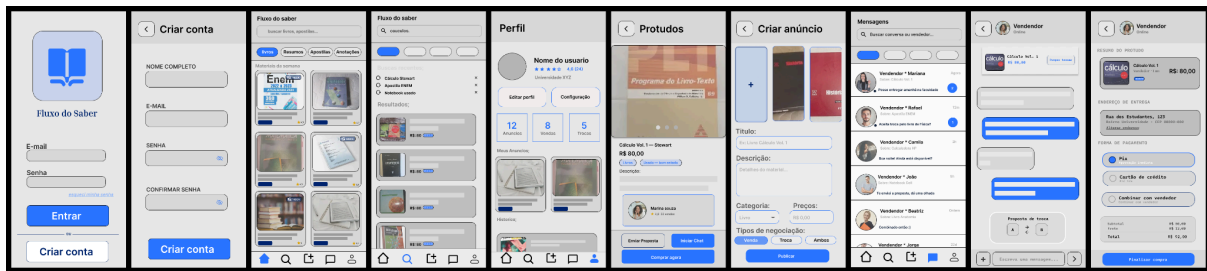
Acessibilidade e Inclusão (WCAG AA)

A interface foi desenvolvida incorporando padrões nativos de acessibilidade web em seus componentes, visando conformidade técnica com as diretrizes do WCAG nível AA:

- **Contraste Mínimo:** A paleta de cores foi planejada para não causar fadiga visual ou dificultar a leitura de usuários com baixa visão. O uso de texto em tons escuros puros (Slate `#0F172A`) sobre fundos claros (Mint `#CFEFEC` e Branco `#FFFFFF`) assegura uma taxa de contraste superior à exigência mínima de 4.5:1 estabelecida pelo padrão AA.
- **Conteúdo Não Textual para Leitores de Tela:** Elementos interativos que utilizam exclusivamente componentes visuais (como o botão contendo o ícone `<ChevronLeft>` para Voltar ou o ícone de `<Trash2>` para Excluir) foram estruturados semanticamente com o atributo `aria-label` (ex: `aria-label="Excluir anúncio"`). Isso assegura que softwares leitores de tela interpretem e verbalizem a função exata do botão para usuários cegos ou com deficiências visuais severas.
- **Foco Visível e Navegação:** Utilizando elementos HTML semânticos nativos (como `<button>` e `<input>`), a aplicação assegura que botões e campos de formulário recebam anéis de foco (`focus-rings`) padronizados pelo CSS, permitindo a navegação acessível inteiramente por teclado.

Diagramação e Protótipo de telas

Wireframes

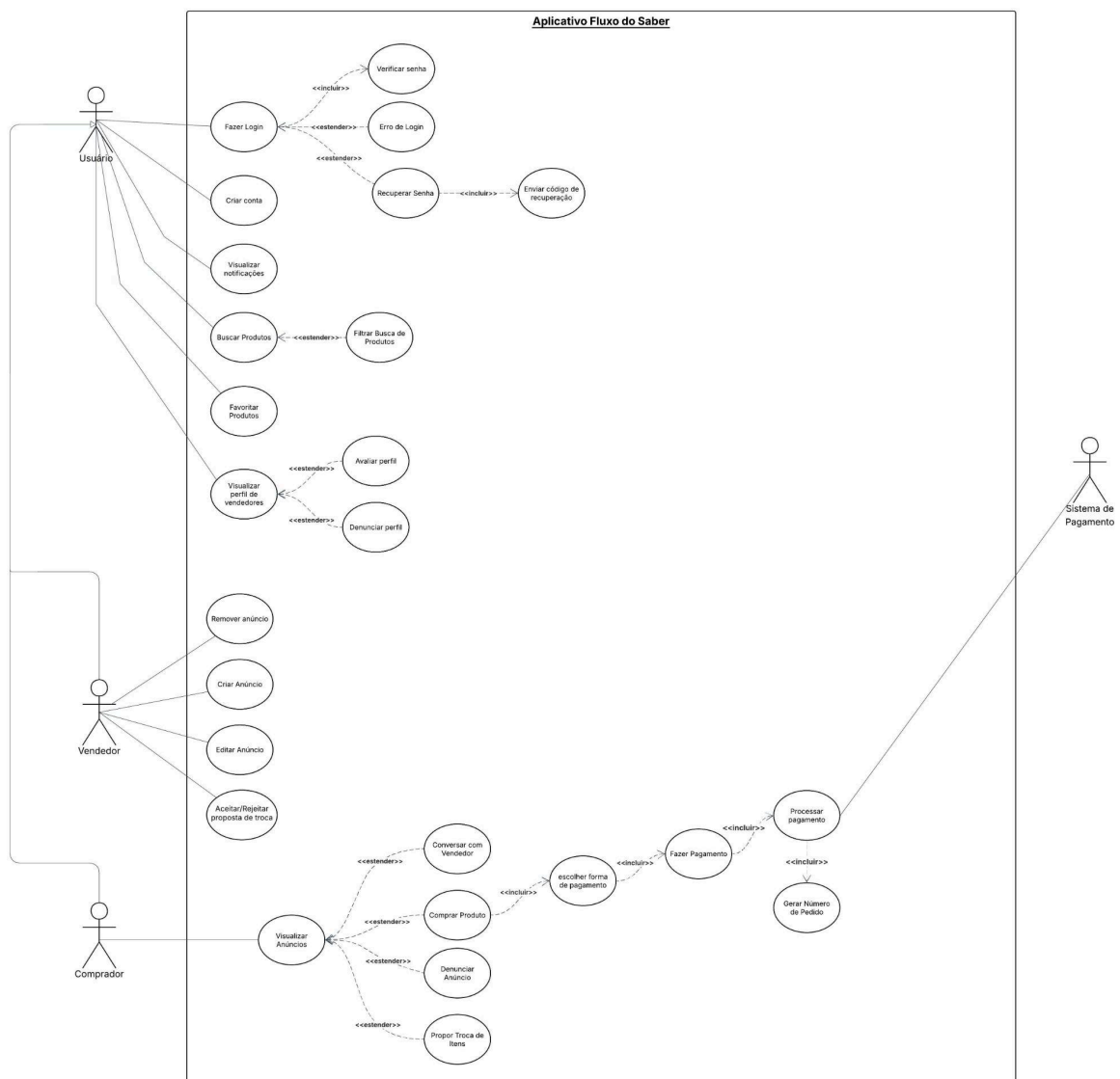


Protótipo de tela

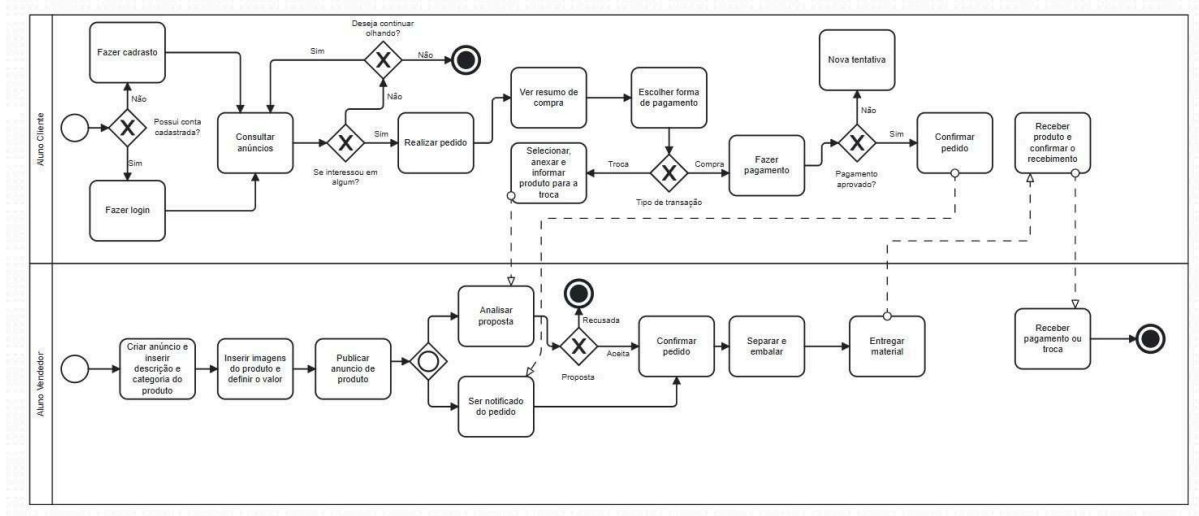
Disponível em [Apresentação Figma](#)

Link do Figma das [telas utilizadas como base](#)

Diagramas



<Diagrama de caso de uso>



<Diagrama BPMN>

Deploy da Aplicação: Vercel

Tela de login: [Login](#)

Tela Home: [Home](#)

Tela de Compra de produto: [Compra](#)

Tela de Criar Anúncio e Troca: [Anúncio e Troca](#)

Tela Notificação: [Notificação](#)

Tela de perfil de vendedor: [Perfil do Vendedor](#)

Tela de perfil do usuário: [Perfil do Usuário](#)

Github do projeto: [Link do Github](#)

Diário de bordo e interações

1. Diário de Bordo e Evolução do Protótipo

- **Estratégia de Desenvolvimento e Diário de Bordo:** A construção da aplicação *Fluxo do Saber* seguiu uma metodologia iterativa fundamentada em ferramentas de Inteligência Artificial. O foco principal ficou no gerenciamento da complexidade lógica e na otimização do consumo de recursos na plataforma de geração de interface (v0).
 - **Integração Multimodal de IAs:** Utilizamos a IA Gemini como facilitador técnico para a estruturação de prompts avançados em inglês. A partir da definição das regras de negócio, o assistente traduzia os requisitos para termos de design e estados de React compatíveis com a ferramenta de prototipagem.
 - **Modularização por Telas Isoladas:** Para minimizar inconsistências no código e excesso de tokens, adotamos uma abordagem de fragmentação. Componentes cruciais como Perfil do Vendedor e Fluxo de Notificações foram gerados de maneira independente, garantindo maior fidelidade visual e funcional para cada módulo específico.
 - **Ajuste Técnico e Refinamento Manual:** Durante a implementação da interface, a equipe, com o suporte do Gemini, realizou intervenções diretas no código gerado pelo v0 para corrigir as referências de imagens (ativos locais). Os caminhos dos arquivos foram ajustados manualmente para apontar para as imagens reais salvas na pasta do nosso projeto (diretório **public**), assegurando a renderização correta.

Por fim, na etapa de integração, consolidamos todos os módulos isolados através de um componente centralizador de navegação, estabelecendo uma jornada de usuário fluida e conectada entre todas as telas do aplicativo.

Apêndice

Adotamos a IA Gemini como ferramenta de apoio técnico para o desenvolvimento em React. A equipe detalhava os requisitos lógicos e visuais em português, e o Gemini auxiliava na elaboração de prompts avançados em inglês para a plataforma de prototipagem (V0). Durante esse processo, aplicamos técnicas de *Role-playing* (atribuição de persona) e *Multimodalidade* (análise de imagens).

Abaixo estão os 5 principais prompts utilizados, demonstrando as técnicas aplicadas:

Prompt 1: Tela de login, cadastro e recuperar senha

- **Prompt utilizado:**

"You are a senior full-stack engineer building a complete, highly interactive mobile-first web application.

I have attached an image showing the 4 screens of our "Authentication Module". Please create a modern, high-fidelity app prototype based on the image and the following specifications.

1. App Overview & Goal

App Name: Fluxo do Saber

Core Purpose: A collaborative academic marketplace for university students.

2. Architecture & Tech Stack

Framework: Next.js (App Router), React, and TypeScript.

UI & Styling: Tailwind CSS, shadcn/ui primitives, and Lucide React icons.

State Management: Use a React state (e.g., `const [currentView, setCurrentView] = useState('welcome')`) to handle navigation seamlessly between the 4 screens without needing a real router or backend.

3. Key Feature Requirements (The Auth Flow)

Build a single interactive component that switches between these 4 views based on user clicks, exactly as shown in the image:

View 1 (Welcome/Primeira Tela): Full background image (or a dark/sleek placeholder), title, subtitle, and a bottom container in green with "Fazer Login" (button) and "Ou crie uma conta" (link/arrow).

View 2 (Sign Up/Criar conta): A back arrow (returns to Welcome), Title, Inputs for Name, Email, Password, Confirm Password (with eye icons to toggle visibility), a Terms checkbox, and a "Criar conta" button.

View 3 (Login): A back arrow, Title "Bem-Vindo de volta!", Inputs for Email and Password, a "Manter conectado" checkbox, "Esqueci minha senha" link, and an "Entrar" button.

View 4 (Forgot Password/Recuperar Senha): A green top header, a back arrow, Email input, "Enviar código" button, and an envelope icon at the bottom with instructions.

4. Layout & Visual Guidelines

Color Palette: Primary Action Color is Vibrant Green (#00C896). Dark Slate for text (#0D1B2A). Backgrounds should match the image (mostly white for forms).

Responsive Layout: CRITICAL - Restrict the main container to a max-width (max-w-[400px] mx-auto) and center it on the screen with a subtle shadow and rounded corners to simulate a mobile phone screen on desktop browsers. Make it min-h-[800px] to feel like a real device.

5. Execution

Do not wait for my approval. Generate the fully functional, interactive code immediately. Ensure all "Back" arrows work, and the links navigate correctly between the forms."

Comentário: Este prompt marcou a configuração inicial do projeto. Utilizamos as imagens dos nossos wireframes (Figma) como base. Com o suporte do Gemini, implementamos a lógica de estado (`currentView`) diretamente na instrução, permitindo a simulação de transições de tela interativas sem a necessidade de construir um backend real.

Prompt 2: Tela principal do aplicativo.

- **Prompt utilizado:**

"You are a senior front-end engineer building "Module 2: Discovery & Shopping" for our mobile-first academic marketplace app.

I have attached 3 reference screens: Home, Search, and Product Details. Please create a single interactive prototype based exactly on these images.

1. Strict UI Constraints

FORCE LIGHT MODE: Absolutely NO dark mode classes.

Mobile Container: Wrap the entire app in a container: `max-w-[400px] mx-auto min-h-[800px] bg-[#CFEFEC] shadow-2xl relative overflow-hidden rounded-[2.5rem] border-[6px] border-gray-900`.

Colors: The primary brand green is #00C896. The background for most of the body is light mint #CFEFEC. All main text must be dark text-slate-900.

2. State & Navigation Logic

Use a React state (e.g., `const [currentView, setCurrentView] = useState('home')`) to toggle between 'home', 'search', and 'product'.

Clicking the search bar on the Home screen MUST change the state to 'search'. Clicking "Cancelar" on the Search screen goes back to 'home'.

Clicking the "Notebook Dell" product card MUST change the state to 'product'. Clicking the back arrow on the Product Details screen goes back to 'home'.

3. View 1: Home Dashboard

Header: Solid green #00C896 top section with Logo, notification bell, and a white Search input placeholder.

Categories (Horizontal Scroll): A flex-row overflow-x-auto hide-scrollbar container of circular icons with text below. Include these exact 6 categories to demonstrate scrolling: Livros e materiais, Eletrônicos, Acessórios, Itens pessoais, Equipamentos técnicos, Lazer.

Product Grids (Horizontal Scroll): The "Em destaque" and "Recomendações para você" sections MUST also scroll horizontally. Cards have a white background, rounded corners, heart icon, product image, title, and price.

Mock Data: Populate with these exact items to show off the scroll: "Livro Física Básica" (R\$ 80,00), "Notebook Dell" (R\$ 1.200,00), "Calculadora Casio" (R\$ 120,00), "Garrafa Contigo" (R\$ 30,00), "Bugiganga Tecnológica" (R\$ 150,00).

Bottom Navigation: A fixed bottom bar bg-[#00C896] z-50 with 5 dark icons. The center button (+) MUST be a prominent, large floating circle sticking out above the bar with a light gradient/shadow and a large black '+'.

4. View 2: Search Screen

Header: Solid green #00C896 top bar with a focused search input and a "Cancelar" text button.

Body: Lists for "Buscas recentes" and "Sugestões" with clock icons on the left of each item.

Keyboard Mockup: Add a static CSS/HTML visual mockup of a mobile keyboard at the absolute bottom of this view to make it look realistic.

5. View 3: Product Details

Header: Solid green #00C896 with Back arrow, Heart, and Share icons.

Image: Large product image area (white or transparent background).

Info: Title ("Notebook Dell") and Price ("R\$ 1.200,00") in large, bold dark text.

Seller Box: A distinct light blue/gray banner showing the seller's avatar, name ("Irinel Silva"), and a 4.5 star rating.

Action Buttons: Three solid green buttons at the bottom ("Conversar", "Propor Troca", "Comprar"). Ensure text inside these buttons is dark black text-slate-900 font-bold. "

Comentário: O objetivo deste prompt foi estruturar a interface principal, incluindo a rolagem horizontal e um botão de ação flutuante. Para gerenciar a densidade visual da tela, o Gemini auxiliou na definição das "Strict UI Constraints" (restrições rigorosas), o que garantiu a fidelidade à nossa paleta de cores e o bloqueio automático de temas escuros indesejados.

Prompt 3: Tela de notificações e trocas recebidas

- **Prompt utilizado:**

"You are a senior UI/UX designer and front-end engineer. We are building a mobile-first academic marketplace. I need you to create the "Notifications & Trade Decision" flow from scratch. We don't have a mockup, so you MUST strictly follow this Design System:

1. Strict UI Constraints

FORCE LIGHT MODE: Absolutely NO dark mode classes.

Mobile Container: Wrap the entire app in exactly this container: className="w-full max-w-[375px] h-[812px] max-h-[812px] mx-auto bg-[#CFEFEC] shadow-2xl relative overflow-hidden rounded-[2.5rem] border-[6px] border-gray-900 flex flex-col".

Colors: Primary Green #00C896. Dark Text #0F172A. Mint Background #CFEFEC. Danger Red #EF4444.

2. State Logic

```
const [view, setView] = useState('notifications') // views: 'notifications', 'decision'
```

```
const [modal, setModal] = useState(null) // modals: 'accepted', 'rejected'
```

3. View Details

View 1: 'notifications'

Header: Green #00C896 background, Back Arrow (hover:bg-black/10 rounded-full p-2), Title "Notificações" (centered, bold).

Body: A list of notifications. Create a standout unread notification card (bg-white shadow-sm rounded-2xl p-4 flex gap-4 items-center cursor-pointer hover:bg-slate-50).

Inside this card: A blue/green dot indicator, a User Avatar, and text: "João Carlos enviou uma proposta de troca para o seu Livro de Física".

Clicking this card sets view to 'decision'.

View 2: 'decision' (The Trade Proposal)

Header: Green background, Back Arrow (goes back to notifications), Title "Proposta de Troca".

Body:

User Info: Avatar of João Carlos + text "João Carlos propôs uma troca!".

The Comparison Card: A large white container bg-white rounded-[2rem] p-5 shadow-sm.

Top Section (Your item): Image placeholder, Title "Livro de Física", label "Seu item".

Middle Section: A circular icon with exchange arrows (ArrowRightLeft from lucide-react) inside a mint #CFEFEC circle, positioned to look like it connects the two items.

Bottom Section (João's item): Image placeholder, Title "Calculadora Científica", label "Item oferecido".

Message box: A grey bubble with a message from João: "Oi! Aceita trocar pela minha calculadora? Está novinha."

Bottom Action Bar: Two large buttons side-by-side.

Left: "Recusar" (Outlined red button, text #EF4444, border #EF4444). Sets modal to 'rejected'.

Right: "Aceitar" (Solid green button #00C896, white text). Sets modal to 'accepted'.

4. Modals (Overlays)

Use backdrop-blur-sm bg-black/40 animate-in fade-in duration-200.

White card rounded-[2rem] p-8 text-center animate-in zoom-in-95 duration-200.

If 'accepted': Green Check circle, Text "Troca Aceita!".

If 'rejected': Red X circle, Text "Proposta Recusada".

Use a `useEffect` to clear modal and go to 'notifications' after 2000ms.

Make it look incredibly polished, matching a premium iOS app."

Comentário: Nesta etapa, não possuíamos wireframes de referência. Descrevemos a regra de negócio para o Gemini, que estruturou o prompt detalhando a hierarquia dos componentes (como a disposição dos botões 'Aceitar' e 'Recusar'). Adicionalmente, foi sugerida a inclusão do hook `useEffect` na instrução para gerenciar o fechamento automático da notificação de sucesso após 2 segundos.

Prompt 4: Checkout e Chat

- **Prompt utilizado:**

"You are a senior front-end engineer. I have attached 5 reference images representing "Module 3 & 4: Checkout Flow and Chat" for our mobile-first academic marketplace app.

Please build a single interactive React component that seamlessly navigates through these 5 views.

1. Strict UI Constraints

FORCE LIGHT MODE: Absolutely NO dark mode classes.

Mobile Container: Wrap the entire app in exactly this container: `className="w-full max-w-[375px] h-[812px] max-h-[812px] mx-auto bg-[#CFEFEC] shadow-2xl relative overflow-hidden rounded-[2.5rem] border-[6px] border-gray-900 flex flex-col"`.

Colors: Primary green #00C896. Text is dark #0F172A. Backgrounds alternate between white and light mint #CFEFEC.

2. State & Navigation Logic

Use `const [view, setView] = useState('summary')` to handle navigation between: 'summary', 'payment', 'processing', 'success', and 'chat'.

3. View Details

View 1: Summary ('summary') -> Header "Resumo da compra". Product details (Notebook Dell, Irinel Silva). Cost breakdown showing "Retirada" as "Presencial (Grátis)" and Total as R\$ 1.200,00. Bottom section with delivery info and a green "Continuar" button (sets view to 'payment').

View 2: Payment ('payment') -> Options for "Pix" and "Cartão de crédito" with radio buttons. A large green "Pagar R\$ 1.200,00" button at the bottom (sets view to 'processing').

View 3: Processing ('processing') -> Centered loading spinner and "Processando Pagamento...". CRITICAL: Use a useEffect to automatically change the view to 'success' after 2500ms.

View 4: Success ('success') -> Green checkmark circle. "Pagamento Aprovado" text. Order ID. A green button "Combinar entrega no chat" (sets view to 'chat').

View 5: Chat ('chat') -> Header with Seller avatar, name ("Irinel Silva") and a phone icon. Chat bubbles alternating sides (white for received, green for sent). Bottom input area with a text field, an attachment icon, a mic icon, and an explicit floating badge/button above it saying "Propor nova oferta" with a tag icon.

Implement the component now with high fidelity to the layout, spacing, and styling of the images."

Comentário: A complexidade deste prompt consistiu em gerenciar múltiplos estados de tela (Resumo, Pagamento, Processamento, Sucesso e Chat) dentro de um único componente React. Com a orientação técnica do Gemini, estruturamos a lógica de transição no prompt.

Prompt 5: Criar anúncio e propor troca.

- **Prompt utilizado:**

"You are a senior front-end engineer. I have attached 4 reference images showing the "Module 5: Create Ad & Propose Trade" flows for our mobile-first academic marketplace app.

Please build a single interactive React component that implements these two flows.

1. Strict UI Constraints

FORCE LIGHT MODE: Absolutely NO dark mode classes (dark:text-white, dark:bg-gray-800, etc.).

Mobile Container: Wrap the entire app in exactly this container: `className="w-full max-w-[375px] h-[812px] max-h-[812px] mx-auto bg-[#CFEFEC] shadow-2xl relative overflow-hidden rounded-[2.5rem] border-[6px] border-gray-900 flex flex-col"`.

Colors: Primary green #00C896. Text #0F172A. Background #CFEFEC.

2. State & Navigation Logic

Use `const [view, setView] = useState('menu')` to handle navigation: 'menu', 'create', 'trade'.

Use `const [modal, setModal] = useState(null)` for the success overlays: 'success-create' and 'success-trade'.

View 'menu': A simple temporary start screen with two large buttons: "Testar Novo Anúncio" and "Testar Propor Troca" to allow testing both flows.

3. View Details

View 'create' (Novo Anúncio):

Header: Back arrow (with `hover:bg-black/10 rounded-full p-2`), "Novo Anúncio" text, green background.

Body: 3 square image placeholders with a camera icon in a horizontal row.

Inputs: "Título do anúncio" (text), "Categoria" (select dropdown), "Tipo do anúncio" (two radio buttons side-by-side: "Venda" and "Troca"). Add subtle focus rings to inputs.

Bottom: Green "Publicar" button. Clicking this sets modal to 'success-create'.

View 'trade' (Propor Troca):

Header: Back arrow, "propor trocar" text, green background.

Body: Shows user's item ("Livro de Física", R\$ 80,00) with an "Editar" text button in green. A thin horizontal divider line. Target item ("Notebook Dell", R\$ 1.200,00) with a "Ver anúncio" text button in green.

Input: "Mensagem (opcional)" textarea with placeholder.

Bottom: Green "Enviar proposta" button. Clicking this sets modal to 'success-trade'.

4. Modals (Overlays)

When modal is active, show an absolute full-screen semi-transparent backdrop (bg-black/50).

Center a white rounded square containing: a large green circle with a checkmark, and the bold text "Anúncio Publicado" (if success-create) or "Proposta enviada" (if success-trade).

Use a `useEffect` to automatically clear the modal (`setModal(null)`) and go back to 'menu' (`setView('menu')`) after 2000ms.

Implement with high fidelity to the layout, spacing, and styling of the provided images."

Comentário: A necessidade de validar múltiplos fluxos simultaneamente exigiu otimização. A estratégia sugerida pelo Gemini consistiu em instruir o v0 a criar um "menu temporário" de acesso, permitindo testar os formulários de criação de anúncio e propostas de troca no mesmo componente. Essa abordagem validou a lógica e reduziu expressivamente o consumo de tokens.

Reflexão sobre Limitações e Alucinações:

1. **Alucinação de Imagens :** Ao gerar novos componentes, o V0 frequentemente perdia o contexto do diretório, inserindo caminhos de imagens genéricos (ex: `/placeholder.svg`) ou errando o apontamento. Com o auxílio do Gemini, o código gerado pelo V0 foi lido e alterado para funcionar como deveria.
2. **Botões Inativos:** Muitas vezes a interface era gerada perfeitamente na parte visual, mas a setinha de "voltar" não funcionava. Então copiávamos o código, colávamos no Gemini e ele nos mostrava qual linha alterar para adicionar propriedades (como `onBack`) que faziam a tela realmente voltar.
3. **Limite de Memória(Tokens):** Prompts excessivamente longos, contendo o escopo do aplicativo inteiro, geravam telas pela metade. A solução foi modularizar a aplicação: gerar os fluxos em prompts separados e, posteriormente, juntar tudo em uma coisa só.